

```

#!/usr/bin/env python3
"""
Comprehensive Technology Database System for Autonomous R&D Engine
Catalogs all technologies, systems, applications, programs, repositories,
app stacks, codes, and scripts
"""

import json
import sqlite3
import datetime
from typing import Dict, List, Any, Optional, Tuple
from dataclasses import dataclass, asdict
from enum import Enum
import hashlib

class TechnologyCategory(Enum):
    HARDWARE = "hardware"
    SOFTWARE = "software"
    FRAMEWORK = "framework"
    LANGUAGE = "language"
    DATABASE = "database"
    CLOUD_SERVICE = "cloud_service"
    API = "api"
    PROTOCOL = "protocol"
    ALGORITHM = "algorithm"
    TOOL = "tool"
    PLATFORM = "platform"
    LIBRARY = "library"
    OPERATING_SYSTEM = "operating_system"
    BLOCKCHAIN = "blockchain"
    AI_ML = "ai_ml"
    MOBILE = "mobile"
    WEB = "web"
    SECURITY = "security"
    NETWORKING = "networking"
    STORAGE = "storage"

class MaturityLevel(Enum):
    EXPERIMENTAL = "experimental"
    ALPHA = "alpha"
    BETA = "beta"
    STABLE = "stable"
    MATURE = "mature"
    LEGACY = "legacy"
    DEPRECATED = "deprecated"

class LicenseType(Enum):
    OPEN_SOURCE = "open_source"
    PROPRIETARY = "proprietary"
    FREEMIUM = "freemium"
    COMMERCIAL = "commercial"
    MIT = "mit"
    GPL = "gpl"
    APACHE = "apache"

```

```
BSD = "bsd"
```

```
@dataclass
class Technology:
    """Represents a single technology component"""
    id: str
    name: str
    category: TechnologyCategory
    description: str
    version: str
    release_date: str
    maturity_level: MaturityLevel
    license_type: LicenseType
    vendor: str
    website: str
    repository: str
    documentation: str
    api_endpoints: List[str]
    dependencies: List[str]
    compatible_with: List[str]
    programming_languages: List[str]
    platforms: List[str]
    use_cases: List[str]
    performance_metrics: Dict[str, Any]
    cost_model: Dict[str, Any]
    security_features: List[str]
    scalability_limits: Dict[str, Any]
    integration_complexity: int # 1-10 scale
    learning_curve: int # 1-10 scale
    community_size: int
    github_stars: int
    last_updated: str
    tags: List[str]
```

```
@dataclass
class AppStack:
    """Represents a complete application stack"""
    id: str
    name: str
    description: str
    stack_type: str # web, mobile, ai, blockchain, etc.
    frontend_technologies: List[str]
    backend_technologies: List[str]
    database_technologies: List[str]
    cloud_services: List[str]
    devops_tools: List[str]
    monitoring_tools: List[str]
    security_tools: List[str]
    total_cost_estimate: Dict[str, float]
    development_time_estimate: int # in weeks
    team_size_requirement: int
    complexity_score: int # 1-100
    scalability_score: int # 1-100
    performance_score: int # 1-100
```

```

        security_score: int # 1-100
        maintainability_score: int # 1-100
        use_cases: List[str]
        success_stories: List[str]
        created_date: str

@dataclass
class CodeRepository:
    """Represents a code repository with reusable components"""
    id: str
    name: str
    repository_url: str
    description: str
    primary_language: str
    languages_used: List[str]
    framework: str
    license: str
    stars: int
    forks: int
    contributors: int
    last_commit: str
    issues_count: int
    pull_requests: int
    code_quality_score: int # 1-100
    documentation_quality: int # 1-100
    test_coverage: float # percentage
    dependencies_count: int
    security_vulnerabilities: int
    performance_benchmarks: Dict[str, Any]
    deployment_instructions: str
    api_documentation: str
    examples: List[str]
    tags: List[str]

class TechnologyDatabase:
    """Main database system for technology catalog"""

    def __init__(self, db_path: str = "technology_database.db"):
        self.db_path = db_path
        self.init_database()
        self.load_comprehensive_catalog()

    def init_database(self):
        """Initialize SQLite database with all necessary tables"""
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        # Technologies table
        cursor.execute('''
            CREATE TABLE IF NOT EXISTS technologies (
                id TEXT PRIMARY KEY,
                name TEXT NOT NULL,
                category TEXT NOT NULL,
                description TEXT,

```

```

        version TEXT,
        release_date TEXT,
        maturity_level TEXT,
        license_type TEXT,
        vendor TEXT,
        website TEXT,
        repository TEXT,
        documentation TEXT,
        api_endpoints TEXT,
        dependencies TEXT,
        compatible_with TEXT,
        programming_languages TEXT,
        platforms TEXT,
        use_cases TEXT,
        performance_metrics TEXT,
        cost_model TEXT,
        security_features TEXT,
        scalability_limits TEXT,
        integration_complexity INTEGER,
        learning_curve INTEGER,
        community_size INTEGER,
        github_stars INTEGER,
        last_updated TEXT,
        tags TEXT
    )
'''

# App Stacks table
cursor.execute('''
    CREATE TABLE IF NOT EXISTS app_stacks (
        id TEXT PRIMARY KEY,
        name TEXT NOT NULL,
        description TEXT,
        stack_type TEXT,
        frontend_technologies TEXT,
        backend_technologies TEXT,
        database_technologies TEXT,
        cloud_services TEXT,
        devops_tools TEXT,
        monitoring_tools TEXT,
        security_tools TEXT,
        total_cost_estimate TEXT,
        development_time_estimate INTEGER,
        team_size_requirement INTEGER,
        complexity_score INTEGER,
        scalability_score INTEGER,
        performance_score INTEGER,
        security_score INTEGER,
        maintainability_score INTEGER,
        use_cases TEXT,
        success_stories TEXT,
        created_date TEXT
    )
''')
```

```

# Code Repositories table
cursor.execute('''
    CREATE TABLE IF NOT EXISTS code_repositories (
        id TEXT PRIMARY KEY,
        name TEXT NOT NULL,
        repository_url TEXT,
        description TEXT,
        primary_language TEXT,
        languages_used TEXT,
        framework TEXT,
        license TEXT,
        stars INTEGER,
        forks INTEGER,
        contributors INTEGER,
        last_commit TEXT,
        issues_count INTEGER,
        pull_requests INTEGER,
        code_quality_score INTEGER,
        documentation_quality INTEGER,
        test_coverage REAL,
        dependencies_count INTEGER,
        security_vulnerabilities INTEGER,
        performance_benchmarks TEXT,
        deployment_instructions TEXT,
        api_documentation TEXT,
        examples TEXT,
        tags TEXT
    )
''')

# Technology Relationships table
cursor.execute('''
    CREATE TABLE IF NOT EXISTS technology_relationships (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        source_tech_id TEXT,
        target_tech_id TEXT,
        relationship_type TEXT,
        strength REAL,
        description TEXT,
        FOREIGN KEY (source_tech_id) REFERENCES technologies
(id),
        FOREIGN KEY (target_tech_id) REFERENCES technologies (id)
    )
''')

# Innovation Opportunities table
cursor.execute('''
    CREATE TABLE IF NOT EXISTS innovation_opportunities (
        id TEXT PRIMARY KEY,
        title TEXT NOT NULL,
        description TEXT,
        technology_combination TEXT,
        market_potential INTEGER,

```

```

        technical_feasibility INTEGER,
        development_effort INTEGER,
        risk_level INTEGER,
        estimated_timeline INTEGER,
        required_skills TEXT,
        potential_revenue TEXT,
        competitive_advantage TEXT,
        created_date TEXT
    )
'''

conn.commit()
conn.close()

def load_comprehensive_catalog(self):
    """Load comprehensive technology catalog from timeline and
    research"""

    # Historical Technologies (from timeline)
    historical_techs = [
        Technology(
            id="pascal_calculator_1642",
            name="Pascal's Calculator (Pascaline)",
            category=TechnologyCategory.HARDWARE,
            description="First mechanical calculator invented by
Blaise Pascal",
            version="1.0",
            release_date="1642-06-19",
            maturity_level=MaturityLevel.LEGACY,
            license_type=LicenseType.PROPRIETARY,
            vendor="Blaise Pascal",
            website="",
            repository="",
            documentation="Historical documents",
            api_endpoints=[],
            dependencies=[],
            compatible_with=[],
            programming_languages=[],
            platforms=["Mechanical"],
            use_cases=["Mathematical calculations", "Accounting"],
            performance_metrics={"accuracy": "High for basic
arithmetic"},
            cost_model={"initial_cost": "Very high for the era"},
            security_features=[],
            scalability_limits={"max_operations": "Basic arithmetic
only"},
            integration_complexity=1,
            learning_curve=3,
            community_size=0,
            github_stars=0,
            last_updated="1642-06-19",
            tags=["historical", "mechanical", "calculator",
"arithmetic"]
        ),
    ],

```

```

Technology(
    id="transistor_1947",
    name="Transistor",
    category=TechnologyCategory.HARDWARE,
    description="Semiconductor device for amplifying or
switching electronic signals",
    version="Point-contact",
    release_date="1947-12-23",
    maturity_level=MaturityLevel.MATURE,
    license_type=LicenseType.OPEN_SOURCE,
    vendor="Bell Labs",
    website="",
    repository="",
    documentation="Bell Labs technical papers",
    api_endpoints=[],
    dependencies=[],
    compatible_with=["All electronic devices"],
    programming_languages=[],
    platforms=["Electronic circuits"],
    use_cases=["Signal amplification", "Digital switching",
"Computing"],
    performance_metrics={"switching_speed": "Nanoseconds",
"power_efficiency": "High"},
    cost_model={"manufacturing_cost": "Low at scale"},
    security_features=[],
    scalability_limits={"miniaturization": "Atomic level"},
    integration_complexity=8,
    learning_curve=9,
    community_size=1000000,
    github_stars=0,
    last_updated="2024-01-01",
    tags=["semiconductor", "electronics", "computing",
"foundation"]
),
Technology(
    id="intel_4004_1971",
    name="Intel 4004 Microprocessor",
    category=TechnologyCategory.HARDWARE,
    description="First commercial microprocessor with 2,300
transistors",
    version="4004",
    release_date="1971-11-15",
    maturity_level=MaturityLevel.LEGACY,
    license_type=LicenseType.PROPRIETARY,
    vendor="Intel Corporation",
    website="https://intel.com",
    repository="",
    documentation="Intel technical specifications",
    api_endpoints=[],
    dependencies=["Transistors", "Silicon wafers"],
    compatible_with=["MCS-4 chipset"],
    programming_languages=["Assembly"],
    platforms=["Embedded systems", "Calculators"],

```

```

        use_cases=["Calculators", "Traffic lights", "Simple
automation"],
        performance_metrics={"clock_speed": "740 kHz",
"instruction_set": "46 instructions"},
        cost_model={"unit_cost": "$200 (1971)"},
        security_features=[],
        scalability_limits={"memory_addressing": "4KB"},
        integration_complexity=7,
        learning_curve=8,
        community_size=10000,
        github_stars=0,
        last_updated="1971-11-15",
        tags=["microprocessor", "cpu", "computing", "historical"]
    )
]

```

```

# Modern Programming Languages
modern_languages = [
    Technology(
        id="python_3_12",
        name="Python",
        category=TechnologyCategory.LANGUAGE,
        description="High-level, interpreted programming language
with dynamic semantics",
        version="3.12",
        release_date="2023-10-02",
        maturity_level=MaturityLevel.MATURE,
        license_type=LicenseType.OPEN_SOURCE,
        vendor="Python Software Foundation",
        website="https://python.org",
        repository="https://github.com/python/cpython",
        documentation="https://docs.python.org",
        api_endpoints=["https://pypi.org/simple/"],
        dependencies=["C compiler", "Standard library"],
        compatible_with=["All major platforms"],
        programming_languages=["Python"],
        platforms=["Windows", "macOS", "Linux", "Unix"],
        use_cases=["Web development", "Data science", "AI/ML",
"Automation", "Scientific computing"],
        performance_metrics={"execution_speed": "Interpreted",
"memory_usage": "Moderate"},
        cost_model={"license_cost": "Free", "development_cost":
"Low"},
        security_features=["Sandboxing", "Type hints", "Security
advisories"],
        scalability_limits={"gil_limitation": "Single-threaded
execution"},
        integration_complexity=3,
        learning_curve=4,
        community_size=15000000,
        github_stars=62000,
        last_updated="2024-01-01",
        tags=["programming", "interpreted", "versatile",
"popular"]
    )
]

```



```

    ),
    Technology(
        id="javascript_es2024",
        name="JavaScript",
        category=TechnologyCategory.LANGUAGE,
        description="Dynamic programming language for web
development",
        version="ES2024",
        release_date="2024-06-01",
        maturity_level=MaturityLevel.MATURE,
        license_type=LicenseType.OPEN_SOURCE,
        vendor="Ecma International",
        website="https://ecma-international.org",
        repository="https://github.com/tc39/ecma262",
        documentation="https://developer.mozilla.org/en-
US/docs/Web/JavaScript",
        api_endpoints=["https://registry.npmjs.org"],
        dependencies=["JavaScript engine"],
        compatible_with=["All web browsers", "Node.js", "Deno",
"Bun"],
        programming_languages=["JavaScript"],
        platforms=["Web browsers", "Server-side", "Mobile",
"Desktop"],
        use_cases=["Web development", "Server-side development",
"Mobile apps", "Desktop apps"],
        performance_metrics={"execution_speed": "JIT compiled",
"startup_time": "Fast"},
        cost_model={"license_cost": "Free", "development_cost":
"Low"},
        security_features=["Same-origin policy", "Content
Security Policy"],
        scalability_limits={"single_threaded": "Event loop
based"},
        integration_complexity=4,
        learning_curve=5,
        community_size=20000000,
        github_stars=0,
        last_updated="2024-06-01",
        tags=["web", "frontend", "backend", "ubiquitous"]
    ),
    Technology(
        id="rust_1_75",
        name="Rust",
        category=TechnologyCategory.LANGUAGE,
        description="Systems programming language focused on
safety, speed, and concurrency",
        version="1.75",
        release_date="2023-12-28",
        maturity_level=MaturityLevel.STABLE,
        license_type=LicenseType.OPEN_SOURCE,
        vendor="Rust Foundation",
        website="https://rust-lang.org",
        repository="https://github.com/rust-lang/rust",
        documentation="https://doc.rust-lang.org",

```

```

        api_endpoints=["https://crates.io/api/v1/"],
        dependencies=["LLVM", "System libraries"],
        compatible_with=["All major platforms"],
        programming_languages=["Rust"],
        platforms=["Windows", "macOS", "Linux", "WebAssembly"],
        use_cases=["Systems programming", "Web backends",
"Blockchain", "Game engines"],
        performance_metrics={"execution_speed": "Native",
"memory_safety": "Guaranteed"},
        cost_model={"license_cost": "Free", "development_cost":
"Medium"},
        security_features=["Memory safety", "Thread safety",
"Zero-cost abstractions"],
        scalability_limits={"compilation_time": "Slower than
C++"},
        integration_complexity=7,
        learning_curve=8,
        community_size=2500000,
        github_stars=97000,
        last_updated="2023-12-28",
        tags=["systems", "memory-safe", "performance", "modern"]
    )
]

# AI/ML Frameworks
ai_frameworks = [
    Technology(
        id="pytorch_2_1",
        name="PyTorch",
        category=TechnologyCategory.AI_ML,
        description="Open source machine learning framework based
on Torch library",
        version="2.1",
        release_date="2023-10-04",
        maturity_level=MaturityLevel.MATURE,
        license_type=LicenseType.BSD,
        vendor="Meta AI",
        website="https://pytorch.org",
        repository="https://github.com/pytorch/pytorch",
        documentation="https://pytorch.org/docs",
        api_endpoints=["https://download.pytorch.org/whl/"],
        dependencies=["Python", "CUDA (optional)", "cuDNN"],
        compatible_with=["Python", "C++", "ONNX"],
        programming_languages=["Python", "C++"],
        platforms=["Linux", "Windows", "macOS"],
        use_cases=["Deep learning", "Computer vision", "NLP",
"Research"],
        performance_metrics={"training_speed": "High",
"memory_efficiency": "Good"},
        cost_model={"license_cost": "Free", "compute_cost":
"Variable"},
        security_features=["Model encryption", "Secure
aggregation"],
        scalability_limits={"distributed_training": "Excellent"},

```

```

        integration_complexity=6,
        learning_curve=7,
        community_size=5000000,
        github_stars=82000,
        last_updated="2023-10-04",
        tags=["deep-learning", "research", "flexible", "gpu"]
    ),
    Technology(
        id="tensorflow_2_15",
        name="TensorFlow",
        category=TechnologyCategory.AI_ML,
        description="End-to-end open source platform for machine
learning",

        version="2.15",
        release_date="2023-11-13",
        maturity_level=MaturityLevel.MATURE,
        license_type=LicenseType.APACHE,
        vendor="Google",
        website="https://tensorflow.org",
        repository="https://github.com/tensorflow/tensorflow",
        documentation="https://tensorflow.org/api_docs",
        api_endpoints=["https://pypi.org/simple/tensorflow/"],
        dependencies=["Python", "CUDA", "cuDNN", "TensorRT"],
        compatible_with=["Python", "JavaScript", "Swift", "C++"],
        programming_languages=["Python", "JavaScript", "C++",
"Swift"],

        platforms=["Linux", "Windows", "macOS", "Mobile", "Web"],
        use_cases=["Production ML", "Mobile deployment", "Edge
computing", "Research"],
        performance_metrics={"inference_speed": "Optimized",
"model_size": "Compressed"},
        cost_model={"license_cost": "Free", "cloud_cost": "Pay-
per-use"},
        security_features=["TensorFlow Privacy", "Federated
learning"],

        scalability_limits={"distributed_training": "Excellent"},
        integration_complexity=7,
        learning_curve=8,
        community_size=8000000,
        github_stars=185000,
        last_updated="2023-11-13",
        tags=["production", "mobile", "scalable", "google"]
    ),
    Technology(
        id="huggingface_transformers_4_36",
        name="Hugging Face Transformers",
        category=TechnologyCategory.AI_ML,
        description="State-of-the-art machine learning for
PyTorch, TensorFlow, and JAX",
        version="4.36",
        release_date="2023-12-01",
        maturity_level=MaturityLevel.STABLE,
        license_type=LicenseType.APACHE,
        vendor="Hugging Face",

```

```

        website="https://huggingface.co",
        repository="https://github.com/huggingface/transformers",
        documentation="https://huggingface.co/docs/transformers",
        api_endpoints=["https://huggingface.co/api/",
"https://api-inference.huggingface.co/"],
        dependencies=["PyTorch or TensorFlow", "tokenizers",
"datasets"],
        compatible_with=["PyTorch", "TensorFlow", "JAX", "ONNX"],
        programming_languages=["Python"],
        platforms=["Linux", "Windows", "macOS", "Cloud"],
        use_cases=["NLP", "Computer vision", "Audio processing",
"Multimodal AI"],
        performance_metrics={"model_loading": "Fast",
"inference_speed": "Optimized"},
        cost_model={"license_cost": "Free", "api_cost":
"Freemium"},
        security_features=["Model cards", "Bias detection",
"Safety filters"],
        scalability_limits={"model_size": "Up to 175B
parameters"},
        integration_complexity=4,
        learning_curve=5,
        community_size=3000000,
        github_stars=133000,
        last_updated="2023-12-01",
        tags=["nlp", "transformers", "pretrained", "community"]
    )
]

```

```

# Cloud Platforms
cloud_platforms = [
    Technology(
        id="aws_2024",
        name="Amazon Web Services",
        category=TechnologyCategory.CLOUD_SERVICE,
        description="Comprehensive cloud computing platform",
        version="2024",
        release_date="2006-03-14",
        maturity_level=MaturityLevel.MATURE,
        license_type=LicenseType.COMMERCIAL,
        vendor="Amazon",
        website="https://aws.amazon.com",
        repository="https://github.com/aws",
        documentation="https://docs.aws.amazon.com",
        api_endpoints=["https://aws.amazon.com/api/"],
        dependencies=["Internet connectivity"],
        compatible_with=["All major platforms and languages"],
        programming_languages=["All"],
        platforms=["Cloud"],
        use_cases=["Web hosting", "Data storage", "Machine
learning", "Enterprise applications"],
        performance_metrics={"uptime": "99.99%",
"global_regions": "31"},
    )
]

```

```

        cost_model={"pricing": "Pay-as-you-go", "free_tier": "12
months"},
        security_features=["IAM", "VPC", "Encryption",
"Compliance certifications"],
        scalability_limits={"virtually_unlimited": "Auto-
scaling"},
        integration_complexity=8,
        learning_curve=9,
        community_size=10000000,
        github_stars=0,
        last_updated="2024-01-01",
        tags=["cloud", "scalable", "enterprise", "comprehensive"]
    ),
    Technology(
        id="azure_2024",
        name="Microsoft Azure",
        category=TechnologyCategory.CLOUD_SERVICE,
        description="Cloud computing service for building,
testing, deploying applications",
        version="2024",
        release_date="2010-02-01",
        maturity_level=MaturityLevel.MATURE,
        license_type=LicenseType.COMMERCIAL,
        vendor="Microsoft",
        website="https://azure.microsoft.com",
        repository="https://github.com/Azure",
        documentation="https://docs.microsoft.com/azure",
        api_endpoints=["https://management.azure.com/"],
        dependencies=["Internet connectivity"],
        compatible_with=["Windows", "Linux", ".NET", "Java",
"Python"],
        programming_languages=["All"],
        platforms=["Cloud", "Hybrid"],
        use_cases=["Enterprise applications", "AI/ML", "DevOps",
"Hybrid cloud"],
        performance_metrics={"uptime": "99.95%",
"global_regions": "60+"},
        cost_model={"pricing": "Pay-as-you-go", "free_tier": "12
months"},
        security_features=["Azure AD", "Key Vault", "Security
Center"],
        scalability_limits={"enterprise_grade": "Auto-scaling"},
        integration_complexity=8,
        learning_curve=8,
        community_size=8000000,
        github_stars=0,
        last_updated="2024-01-01",
        tags=["cloud", "microsoft", "enterprise", "hybrid"]
    ),
    Technology(
        id="gcp_2024",
        name="Google Cloud Platform",
        category=TechnologyCategory.CLOUD_SERVICE,
        description="Suite of cloud computing services",

```

```

        version="2024",
        release_date="2008-04-07",
        maturity_level=MaturityLevel.MATURE,
        license_type=LicenseType.COMMERCIAL,
        vendor="Google",
        website="https://cloud.google.com",
        repository="https://github.com/GoogleCloudPlatform",
        documentation="https://cloud.google.com/docs",
        api_endpoints=["https://cloud.google.com/apis"],
        dependencies=["Internet connectivity"],
        compatible_with=["All major platforms and languages"],
        programming_languages=["All"],
        platforms=["Cloud"],
        use_cases=["Data analytics", "Machine learning",
"Kubernetes", "Big data"],
        performance_metrics={"uptime": "99.95%",
"global_regions": "35+"},
        cost_model={"pricing": "Pay-as-you-go",
"sustained_use_discounts": "Yes"},
        security_features=["IAM", "VPC", "Cloud Security Command
Center"],

        scalability_limits={"google_scale": "Petabyte scale"},
        integration_complexity=7,
        learning_curve=7,
        community_size=5000000,
        github_stars=0,
        last_updated="2024-01-01",
        tags=["cloud", "google", "analytics", "kubernetes"]
    )
]

```

```

# Web Frameworks
web_frameworks = [
    Technology(
        id="nextjs_14",
        name="Next.js",
        category=TechnologyCategory.FRAMEWORK,
        description="React framework for production with hybrid
static & server rendering",
        version="14.0",
        release_date="2023-10-26",
        maturity_level=MaturityLevel.STABLE,
        license_type=LicenseType.MIT,
        vendor="Vercel",
        website="https://nextjs.org",
        repository="https://github.com/vercel/next.js",
        documentation="https://nextjs.org/docs",
        api_endpoints=["https://registry.npmjs.org/next"],
        dependencies=["React", "Node.js", "Webpack"],
        compatible_with=["React", "TypeScript", "Tailwind CSS"],
        programming_languages=["JavaScript", "TypeScript"],
        platforms=["Web", "Server"],
        use_cases=["Web applications", "Static sites", "E-
commerce", "Dashboards"],
    )
]

```

```

        performance_metrics={"build_time": "Fast",
"runtime_performance": "Optimized"},
        cost_model={"license_cost": "Free", "hosting_cost":
"Variable"},
        security_features=["CSRF protection", "XSS protection",
"Content Security Policy"],
        scalability_limits={"serverless_ready": "Excellent"},
        integration_complexity=5,
        learning_curve=6,
        community_size=3000000,
        github_stars=124000,
        last_updated="2023-10-26",
        tags=["react", "ssr", "static", "vercel"]
    ),
    Technology(
        id="fastapi_0_104",
        name="FastAPI",
        category=TechnologyCategory.FRAMEWORK,
        description="Modern, fast web framework for building APIs
with Python",
        version="0.104",
        release_date="2023-09-17",
        maturity_level=MaturityLevel.STABLE,
        license_type=LicenseType.MIT,
        vendor="Sebastián Ramirez",
        website="https://fastapi.tiangolo.com",
        repository="https://github.com/tiangolo/fastapi",
        documentation="https://fastapi.tiangolo.com",
        api_endpoints=["https://pypi.org/simple/fastapi/"],
        dependencies=["Python", "Starlette", "Pydantic"],
        compatible_with=["Python", "OpenAPI", "JSON Schema"],
        programming_languages=["Python"],
        platforms=["Server", "Cloud"],
        use_cases=["REST APIs", "GraphQL", "WebSocket",
"Microservices"],
        performance_metrics={"request_throughput": "High",
"response_time": "Low"},
        cost_model={"license_cost": "Free", "hosting_cost":
"Low"},
        security_features=["OAuth2", "JWT", "CORS", "Input
validation"],
        scalability_limits={"async_support": "Excellent"},
        integration_complexity=4,
        learning_curve=5,
        community_size=1500000,
        github_stars=76000,
        last_updated="2023-09-17",
        tags=["python", "api", "async", "openapi"]
    )
]

```

```

# Databases
databases = [
    Technology(

```

```

        id="postgresql_16",
        name="PostgreSQL",
        category=TechnologyCategory.DATABASE,
        description="Advanced open source relational database",
        version="16.0",
        release_date="2023-09-14",
        maturity_level=MaturityLevel.MATURE,
        license_type=LicenseType.OPEN_SOURCE,
        vendor="PostgreSQL Global Development Group",
        website="https://postgresql.org",
        repository="https://github.com/postgres/postgres",
        documentation="https://postgresql.org/docs",
        api_endpoints=["SQL interface", "libpq"],
        dependencies=["Operating system", "C compiler"],
        compatible_with=["All major platforms", "SQL standard"],
        programming_languages=["SQL", "PL/pgSQL", "Python",
"JavaScript"],
        platforms=["Linux", "Windows", "macOS", "Unix"],
        use_cases=["Web applications", "Data warehousing", "GIS",
"Analytics"],
        performance_metrics={"query_performance": "High",
"concurrent_connections": "Thousands"},
        cost_model={"license_cost": "Free", "support_cost":
"Optional"},
        security_features=["SSL/TLS", "Row-level security",
"Encryption"],
        scalability_limits={"table_size": "32TB",
"database_size": "Unlimited"},
        integration_complexity=6,
        learning_curve=7,
        community_size=2000000,
        github_stars=16000,
        last_updated="2023-09-14",
        tags=["relational", "acid", "extensible", "standards"]
    ),
    Technology(
        id="mongodb_7_0",
        name="MongoDB",
        category=TechnologyCategory.DATABASE,
        description="Document-oriented NoSQL database",
        version="7.0",
        release_date="2023-08-29",
        maturity_level=MaturityLevel.MATURE,
        license_type=LicenseType.OPEN_SOURCE,
        vendor="MongoDB Inc.",
        website="https://mongodb.com",
        repository="https://github.com/mongodb/mongo",
        documentation="https://docs.mongodb.com",
        api_endpoints=["MongoDB Wire Protocol", "REST API"],
        dependencies=["Operating system"],
        compatible_with=["All major platforms", "JSON/BSON"],
        programming_languages=["JavaScript", "Python", "Java",
"C#"],
        platforms=["Linux", "Windows", "macOS", "Cloud"],

```



```

        use_cases=["Web applications", "Content management",
"IoT", "Real-time analytics"],
        performance_metrics={"read_performance": "High",
"write_performance": "High"},
        cost_model={"license_cost": "Free/Commercial",
"cloud_cost": "Pay-per-use"},
        security_features=["Authentication", "Authorization",
"Encryption"],
        scalability_limits={"horizontal_scaling": "Excellent"},
        integration_complexity=5,
        learning_curve=6,
        community_size=3000000,
        github_stars=26000,
        last_updated="2023-08-29",
        tags=["nosql", "document", "scalable", "flexible"]
    )
]

```

```

# Blockchain Technologies
blockchain_techs = [
    Technology(
        id="ethereum_2024",
        name="Ethereum",
        category=TechnologyCategory.BLOCKCHAIN,
        description="Decentralized platform for smart contracts
and DApps",
        version="2.0",
        release_date="2015-07-30",
        maturity_level=MaturityLevel.MATURE,
        license_type=LicenseType.OPEN_SOURCE,
        vendor="Ethereum Foundation",
        website="https://ethereum.org",
        repository="https://github.com/ethereum/go-ethereum",
        documentation="https://ethereum.org/developers/docs",
        api_endpoints=["JSON-RPC", "Web3 API"],
        dependencies=["Consensus mechanism", "Network nodes"],
        compatible_with=["Web3", "Solidity", "EVM"],
        programming_languages=["Solidity", "Vyper",
"JavaScript"],
        platforms=["Blockchain", "Web3"],
        use_cases=["Smart contracts", "DeFi", "NFTs", "DAOs"],
        performance_metrics={"tps": "15", "block_time": "12
seconds"},
        cost_model={"gas_fees": "Variable", "staking_rewards":
"4-6% APY"},
        security_features=["Proof of Stake", "Cryptographic
hashing", "Decentralization"],
        scalability_limits={"layer2_solutions": "Required for
scale"},
        integration_complexity=8,
        learning_curve=9,
        community_size=5000000,
        github_stars=47000,
        last_updated="2024-01-01",
    )
]

```

```

        tags=["blockchain", "smart-contracts", "defi", "web3"]
    ),
    Technology(
        id="solidity_0_8_23",
        name="Solidity",
        category=TechnologyCategory.LANGUAGE,
        description="Object-oriented programming language for
smart contracts",
        version="0.8.23",
        release_date="2023-11-08",
        maturity_level=MaturityLevel.STABLE,
        license_type=LicenseType.GPL,
        vendor="Ethereum Foundation",
        website="https://soliditylang.org",
        repository="https://github.com/ethereum/solidity",
        documentation="https://docs.soliditylang.org",
        api_endpoints=["Compiler API"],
        dependencies=["Ethereum Virtual Machine"],
        compatible_with=["Ethereum", "EVM-compatible chains"],
        programming_languages=["Solidity"],
        platforms=["Ethereum", "Polygon", "BSC", "Avalanche"],
        use_cases=["Smart contracts", "DeFi protocols", "NFT
contracts", "DAOs"],
        performance_metrics={"compilation_speed": "Fast",
"gas_optimization": "Good"},
        cost_model={"license_cost": "Free", "gas_cost":
"Variable"},
        security_features=["Static analysis", "Formal
verification", "Audit tools"],
        scalability_limits={"contract_size": "24KB limit"},
        integration_complexity=7,
        learning_curve=8,
        community_size=1000000,
        github_stars=23000,
        last_updated="2023-11-08",
        tags=["smart-contracts", "ethereum", "evm", "defi"]
    )
]

# Combine all technologies
all_technologies = (
    historical_techs + modern_languages + ai_frameworks +
    cloud_platforms + web_frameworks + databases +
blockchain_techs
)

# Insert technologies into database
for tech in all_technologies:
    self.add_technology(tech)

# Load app stacks
self.load_app_stacks()

# Load code repositories

```

```

        self.load_code_repositories()

    def add_technology(self, tech: Technology):
        """Add a technology to the database"""
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        cursor.execute('''
            INSERT OR REPLACE INTO technologies VALUES (
                ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?, ?, ?, ?, ?, ?, ?, ?
            )
        ''', (
            tech.id, tech.name, tech.category.value, tech.description,
tech.version,
            tech.release_date, tech.maturity_level.value,
tech.license_type.value,
            tech.vendor, tech.website, tech.repository,
tech.documentation,
            json.dumps(tech.api_endpoints),
json.dumps(tech.dependencies),
            json.dumps(tech.compatible_with),
json.dumps(tech.programming_languages),
            json.dumps(tech.platforms), json.dumps(tech.use_cases),
            json.dumps(tech.performance_metrics),
json.dumps(tech.cost_model),
            json.dumps(tech.security_features),
json.dumps(tech.scalability_limits),
            tech.integration_complexity, tech.learning_curve,
tech.community_size,
            tech.github_stars, tech.last_updated, json.dumps(tech.tags)
        ))

        conn.commit()
        conn.close()

    def load_app_stacks(self):
        """Load predefined application stacks"""
        app_stacks = [
            AppStack(
                id="modern_web_stack_2024",
                name="Modern Web Application Stack",
                description="Full-stack web application with React,
Node.js, and PostgreSQL",
                stack_type="web",
                frontend_technologies=["React", "Next.js", "TypeScript",
"Tailwind CSS"],
                backend_technologies=["Node.js", "Express.js", "FastAPI",
"GraphQL"],
                database_technologies=["PostgreSQL", "Redis",
"Elasticsearch"],
                cloud_services=["AWS", "Vercel", "Supabase"],
                devops_tools=["Docker", "Kubernetes", "GitHub Actions",
"Terraform"],

```

```

        monitoring_tools=["DataDog", "Sentry", "New Relic"],
        security_tools=["Auth0", "AWS IAM", "SSL/TLS"],
        total_cost_estimate={"monthly": 500.0, "annual": 6000.0},
        development_time_estimate=12,
        team_size_requirement=4,
        complexity_score=75,
        scalability_score=90,
        performance_score=85,
        security_score=80,
        maintainability_score=85,
        use_cases=["E-commerce", "SaaS platforms", "Content
management", "Dashboards"],
        success_stories=["Airbnb", "Netflix", "Shopify"],
        created_date="2024-01-01"
    ),
    AppStack(
        id="ai_ml_stack_2024",
        name="AI/ML Development Stack",
        description="Complete stack for AI/ML model development
and deployment",
        stack_type="ai",
        frontend_technologies=["Streamlit", "Gradio", "React",
"Jupyter"],
        backend_technologies=["FastAPI", "Flask", "MLflow",
"Kubeflow"],
        database_technologies=["PostgreSQL", "MongoDB", "Vector
databases"],
        cloud_services=["AWS SageMaker", "Google AI Platform",
"Azure ML"],
        devops_tools=["Docker", "Kubernetes", "MLOps pipelines"],
        monitoring_tools=["Weights & Biases", "Neptune",
"TensorBoard"],
        security_tools=["Model encryption", "Data privacy
tools"],
        total_cost_estimate={"monthly": 2000.0, "annual":
24000.0},
        development_time_estimate=16,
        team_size_requirement=6,
        complexity_score=90,
        scalability_score=95,
        performance_score=90,
        security_score=75,
        maintainability_score=70,
        use_cases=["Computer vision", "NLP", "Recommendation
systems", "Predictive analytics"],
        success_stories=["OpenAI", "Hugging Face", "Scale AI"],
        created_date="2024-01-01"
    ),
    AppStack(
        id="blockchain_dapp_stack_2024",
        name="Blockchain DApp Stack",
        description="Decentralized application development
stack",
        stack_type="blockchain",

```

```

        frontend_technologies=["React", "Web3.js", "Ethers.js",
"MetaMask"],
        backend_technologies=["Node.js", "IPFS", "The Graph",
"Chainlink"],
        database_technologies=["Blockchain", "IPFS", "Arweave"],
        cloud_services=["Infura", "Alchemy", "AWS", "Pinata"],
        devops_tools=["Hardhat", "Truffle", "Remix", "GitHub
Actions"],
        monitoring_tools=["Etherscan", "DeFi Pulse", "Dune
Analytics"],
        security_tools=["OpenZeppelin", "MythX", "Slither"],
        total_cost_estimate={"monthly": 1000.0, "annual":
12000.0},
        development_time_estimate=20,
        team_size_requirement=5,
        complexity_score=95,
        scalability_score=60,
        performance_score=70,
        security_score=90,
        maintainability_score=65,
        use_cases=["DeFi protocols", "NFT marketplaces", "DAOs",
"Gaming"],
        success_stories=["Uniswap", "OpenSea", "Compound"],
        created_date="2024-01-01"
    )
]

```

```

conn = sqlite3.connect(self.db_path)
cursor = conn.cursor()

for stack in app_stacks:
    cursor.execute('''
        INSERT OR REPLACE INTO app_stacks VALUES (
            ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?, ?
        )
    ''', (
        stack.id, stack.name, stack.description,
stack.stack_type,
        json.dumps(stack.frontend_technologies),
        json.dumps(stack.backend_technologies),
        json.dumps(stack.database_technologies),
        json.dumps(stack.cloud_services),
        json.dumps(stack.devops_tools),
        json.dumps(stack.monitoring_tools),
        json.dumps(stack.security_tools),
        json.dumps(stack.total_cost_estimate),
        stack.development_time_estimate,
        stack.team_size_requirement,
        stack.complexity_score, stack.scalability_score,
        stack.performance_score,
        stack.security_score, stack.maintainability_score,
        json.dumps(stack.use_cases),
        json.dumps(stack.success_stories),

```

```

        stack.created_date
    ))

    conn.commit()
    conn.close()

def load_code_repositories(self):
    """Load trending and important code repositories"""
    repositories = [
        CodeRepository(
            id="openscreen_demo_tool",
            name="OpenScreen",

repository_url="https://github.com/siddharthvaddem/openscreen",
            description="Create stunning demos for free. Open-source
alternative to Screen Studio",
            primary_language="TypeScript",
            languages_used=["TypeScript", "JavaScript", "CSS"],
            framework="React",
            license="MIT",
            stars=15227,
            forks=1046,
            contributors=12,
            last_commit="2024-01-01",
            issues_count=45,
            pull_requests=23,
            code_quality_score=85,
            documentation_quality=80,
            test_coverage=75.0,
            dependencies_count=45,
            security_vulnerabilities=0,
            performance_benchmarks={"load_time": "< 2s",
"bundle_size": "< 500KB"},
            deployment_instructions="npm install && npm run build",
            api_documentation="README.md",
            examples=["Demo creation", "Screen recording", "Video
editing"],
            tags=["demo", "screen-recording", "open-source",
"typescript"]
        ),
        CodeRepository(
            id="claude_code_terminal",
            name="Claude Code",
            repository_url="https://github.com/anthropics/claude-
code",
            description="Agentic coding tool that lives in your
terminal",
            primary_language="Shell",
            languages_used=["Shell", "Python", "JavaScript"],
            framework="CLI",
            license="Apache-2.0",
            stars=103699,
            forks=16353,
            contributors=25,

```

```

        last_commit="2024-01-01",
        issues_count=120,
        pull_requests=45,
        code_quality_score=90,
        documentation_quality=95,
        test_coverage=85.0,
        dependencies_count=20,
        security_vulnerabilities=0,
        performance_benchmarks={"response_time": "< 1s",
"accuracy": "95%"},
        deployment_instructions="curl -sSL install.sh | bash",
        api_documentation="docs/api.md",
        examples=["Code generation", "Git workflows", "Terminal
automation"],
        tags=["ai", "coding", "terminal", "automation"]
    ),
    CodeRepository(
        id="timesfm_google",
        name="TimesFM",
        repository_url="https://github.com/google-
research/timesfm",
        description="Time Series Foundation Model for
forecasting",
        primary_language="Python",
        languages_used=["Python", "Jupyter Notebook"],
        framework="TensorFlow",
        license="Apache-2.0",
        stars=12596,
        forks=1039,
        contributors=8,
        last_commit="2024-01-01",
        issues_count=35,
        pull_requests=12,
        code_quality_score=88,
        documentation_quality=85,
        test_coverage=80.0,
        dependencies_count=30,
        security_vulnerabilities=0,
        performance_benchmarks={"accuracy": "SOTA",
"inference_time": "< 100ms"},
        deployment_instructions="pip install timesfm",
        api_documentation="docs/",
        examples=["Time series forecasting", "Financial
prediction", "Demand forecasting"],
        tags=["time-series", "forecasting", "machine-learning",
"google"]
    )
]

conn = sqlite3.connect(self.db_path)
cursor = conn.cursor()

for repo in repositories:
    cursor.execute('')
```

```

        INSERT OR REPLACE INTO code_repositories VALUES (
            ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
            ?, ?, ?, ?, ?, ?
        )
    '''
    (
        repo.id, repo.name, repo.repository_url,
repo.description,
        repo.primary_language, json.dumps(repo.languages_used),
repo.framework,
        repo.license, repo.stars, repo.forks, repo.contributors,
        repo.last_commit, repo.issues_count, repo.pull_requests,
        repo.code_quality_score, repo.documentation_quality,
repo.test_coverage,
        repo.dependencies_count, repo.security_vulnerabilities,
        json.dumps(repo.performance_benchmarks),
repo.deployment_instructions,
        repo.api_documentation, json.dumps(repo.examples),
        json.dumps(repo.tags)
    ))

```

```

    conn.commit()
    conn.close()

```

```

def search_technologies(self, query: str, category: Optional[str] =
None,

```

```

        limit: int = 10) -> List[Dict]:

```

```

    """Search technologies by query and optional category"""

```

```

    conn = sqlite3.connect(self.db_path)

```

```

    cursor = conn.cursor()

```

```

    base_query = '''

```

```

        SELECT * FROM technologies

```

```

        WHERE (name LIKE ? OR description LIKE ? OR tags LIKE ?)

```

```

    '''

```

```

    params = [f'%{query}%', f'%{query}%', f'%{query}%']

```

```

    if category:

```

```

        base_query += ' AND category = ?'

```

```

        params.append(category)

```

```

    base_query += f' LIMIT {limit}'

```

```

    cursor.execute(base_query, params)

```

```

    results = cursor.fetchall()

```

```

    conn.close()

```

```

    # Convert to dictionaries

```

```

    columns = [desc[0] for desc in cursor.description]

```

```

    return [dict(zip(columns, row)) for row in results]

```

```

def get_compatible_technologies(self, tech_id: str) -> List[Dict]:

```

```

    """Get technologies compatible with the given technology"""

```

```

    conn = sqlite3.connect(self.db_path)

```



```

        cursor = conn.cursor()

        # Get the technology's compatible_with list
        cursor.execute('SELECT compatible_with FROM technologies WHERE id
= ?', (tech_id,))
        result = cursor.fetchone()

        if not result:
            conn.close()
            return []

        compatible_with = json.loads(result[0] or '[]')

        if not compatible_with:
            conn.close()
            return []

        # Search for technologies that match the compatible_with list
        placeholders = ','.join(['?' for _ in compatible_with])
        query = f'''
            SELECT * FROM technologies
            WHERE name IN ({placeholders}) OR id IN ({placeholders})
        '''

        cursor.execute(query, compatible_with + compatible_with)
        results = cursor.fetchall()

        conn.close()

        columns = [desc[0] for desc in cursor.description]
        return [dict(zip(columns, row)) for row in results]

def generate_stack_recommendations(self, use_case: str,
                                   complexity_preference: str =
"medium") -> List[Dict]:
    """Generate app stack recommendations based on use case"""
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()

    # Map complexity preference to score ranges
    complexity_ranges = {
        "low": (0, 50),
        "medium": (40, 80),
        "high": (70, 100)
    }

    min_complexity, max_complexity =
complexity_ranges.get(complexity_preference, (40, 80))

    cursor.execute('''
        SELECT * FROM app_stacks
        WHERE use_cases LIKE ?
        AND complexity_score BETWEEN ? AND ?
        ORDER BY scalability_score DESC, performance_score DESC
    ''')

```

```

        '', (f'%{use_case}%', min_complexity, max_complexity))

    results = cursor.fetchall()
    conn.close()

    columns = [desc[0] for desc in cursor.description]
    return [dict(zip(columns, row)) for row in results]

def analyze_technology_trends(self) -> Dict[str, Any]:
    """Analyze technology trends from the database"""
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()

    # Category distribution
    cursor.execute('''
        SELECT category, COUNT(*) as count
        FROM technologies
        GROUP BY category
        ORDER BY count DESC
    ''')
    category_distribution = dict(cursor.fetchall())

    # Maturity level distribution
    cursor.execute('''
        SELECT maturity_level, COUNT(*) as count
        FROM technologies
        GROUP BY maturity_level
        ORDER BY count DESC
    ''')
    maturity_distribution = dict(cursor.fetchall())

    # License type distribution
    cursor.execute('''
        SELECT license_type, COUNT(*) as count
        FROM technologies
        GROUP BY license_type
        ORDER BY count DESC
    ''')
    license_distribution = dict(cursor.fetchall())

    # Top technologies by GitHub stars
    cursor.execute('''
        SELECT name, github_stars
        FROM technologies
        WHERE github_stars > 0
        ORDER BY github_stars DESC
        LIMIT 10
    ''')
    top_starred = cursor.fetchall()

    # Average complexity by category
    cursor.execute('''
        SELECT category, AVG(integration_complexity) as
    avg_complexity,

```

```

        AVG(learning_curve) as avg_learning_curve
    FROM technologies
    GROUP BY category
'''
complexity_by_category = cursor.fetchall()

conn.close()

return {
    "category_distribution": category_distribution,
    "maturity_distribution": maturity_distribution,
    "license_distribution": license_distribution,
    "top_starred_technologies": top_starred,
    "complexity_by_category": complexity_by_category,
    "total_technologies": sum(category_distribution.values()),
    "analysis_date": datetime.datetime.now().isoformat()
}

def export_database(self, format: str = "json") -> str:
    """Export the entire database to JSON or CSV format"""
    conn = sqlite3.connect(self.db_path)

    if format.lower() == "json":
        # Export to JSON
        data = {}

        # Export technologies
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM technologies")
        columns = [desc[0] for desc in cursor.description]
        technologies = [dict(zip(columns, row)) for row in
cursor.fetchall()]
        data["technologies"] = technologies

        # Export app stacks
        cursor.execute("SELECT * FROM app_stacks")
        columns = [desc[0] for desc in cursor.description]
        app_stacks = [dict(zip(columns, row)) for row in
cursor.fetchall()]
        data["app_stacks"] = app_stacks

        # Export repositories
        cursor.execute("SELECT * FROM code_repositories")
        columns = [desc[0] for desc in cursor.description]
        repositories = [dict(zip(columns, row)) for row in
cursor.fetchall()]
        data["code_repositories"] = repositories

        conn.close()

        return json.dumps(data, indent=2)

    conn.close()
    return "Format not supported"

```

```

# Example usage and demonstration
def main():
    """Demonstrate the technology database system"""

    # Initialize database
    print("Initializing Technology Database System...")
    db = TechnologyDatabase()

    # Search for AI technologies
    print("\n=== Searching for AI Technologies ===")
    ai_techs = db.search_technologies("AI", category="ai_ml")
    for tech in ai_techs:
        print(f"- {tech['name']}: {tech['description']}")

    # Get compatible technologies for PyTorch
    print("\n=== Technologies Compatible with PyTorch ===")
    compatible = db.get_compatible_technologies("pytorch_2_1")
    for tech in compatible:
        print(f"- {tech['name']}: {tech['description']}")

    # Generate stack recommendations for web development
    print("\n=== Web Development Stack Recommendations ===")
    stacks = db.generate_stack_recommendations("Web applications",
"medium")
    for stack in stacks:
        print(f"- {stack['name']}: {stack['description']}")
        print(f"  Complexity: {stack['complexity_score']}/100")
        print(f"  Scalability: {stack['scalability_score']}/100")

    # Analyze technology trends
    print("\n=== Technology Trends Analysis ===")
    trends = db.analyze_technology_trends()
    print(f"Total Technologies: {trends['total_technologies']}")
    print("\nCategory Distribution:")
    for category, count in trends['category_distribution'].items():
        print(f"  {category}: {count}")

    print("\nTop Starred Technologies:")
    for name, stars in trends['top_starred_technologies']:
        print(f"  {name}: {stars}, {stars}")

    print("\nTechnology Database System demonstration completed!")

if __name__ == "__main__":
    main()

```